

PySpark Interview Questions — Spark Configuration & Performance

Focused on PySpark, DataFrames, Spark SQL, AQE, joins, partitioning, memory, shuffle, and Structured Streaming. Answers are concise and interview-oriented.

1. Core Configuration & Deployment

1. What is the difference between `spark-submit --conf`, `spark-defaults.conf`, and `SparkSession.conf.set()`?

Answer: `spark-submit` and `spark-defaults.conf` provide deployment/application configuration; `SparkConf/SparkSession` settings can control runtime behavior, but deploy-related settings such as driver/executor resources may need to be supplied before the application starts.

2. Which configuration has higher precedence?

Answer: `SparkConf` values set directly in the application generally take precedence over `--conf/properties-file` values, which take precedence over `spark-defaults.conf`, subject to deployment-specific behavior.

3. How do you inspect Spark configurations in PySpark?

Answer: Use `spark.conf.get('key')` for SQL/runtime configs, and inspect `SparkConf` through `spark.sparkContext.getConf().getAll()`. The Spark UI Environment tab is also useful for explicitly set properties.

4. Why should you specify units such as 4g or 60s?

Answer: Spark configuration values for memory and durations use documented units. Explicit units avoid ambiguity and make configuration portable/readable.

2. Driver & Executor

5. What is `spark.driver.memory`?

Answer: Memory allocated to the Spark driver. It is not the memory available to executors.

6. What is `spark.executor.memory`?

Answer: Heap memory allocated to each executor JVM.

7. What is the difference between executor memory and executor memory overhead?

Answer: `executor.memory` is JVM heap; memory overhead covers non-heap/native/Python-related memory outside the JVM heap. PySpark workloads can be sensitive to overhead.

8. What happens if the driver runs out of memory?

Answer: The driver process can fail, often because of collecting too much data, large query plans, excessive metadata, or driver-side Python/JVM objects.

9. Why is `collect()` dangerous on a large DataFrame?

Answer: `collect()` brings all rows to the driver. It can exhaust driver memory regardless of how large the cluster is.

3. Partitions, Parallelism & Shuffle

10. What does `spark.sql.shuffle.partitions` control?

Answer: The default number of partitions used for shuffle operations in Spark SQL/DataFrame workloads.

11. What is `spark.default.parallelism`?

Answer: A default parallelism setting used by certain RDD operations when the number of partitions is otherwise unspecified.

12. Why can too many partitions hurt performance?

Answer: They increase task scheduling overhead, file/metadata overhead, and potentially create many tiny tasks/files.

13. Why can too few partitions hurt performance?

Answer: Tasks become large, parallelism falls, and individual tasks can become slow or memory-intensive.

14. What is a shuffle?

Answer: A redistribution of records across executors, commonly caused by joins, `groupBy`, `distinct`, repartition, and aggregations.

15. How do you reduce unnecessary shuffle in PySpark?

Answer: Filter early, select only required columns, use appropriate partitioning, broadcast small dimension tables, avoid unnecessary repartition, and leverage AQE.

4. AQE & Performance

16. What is Adaptive Query Execution?

Answer: AQE re-optimizes a query during execution using runtime statistics. It can coalesce shuffle partitions, handle skew, and change join strategies.

17. What does `spark.sql.adaptive.enabled` do?

Answer: Enables Adaptive Query Execution.

18. How does AQE help with too many shuffle partitions?

Answer: It can coalesce post-shuffle partitions so the final number of tasks better matches the actual runtime data size.

19. What is skew join handling?

Answer: AQE can identify skewed shuffle partitions and split/replicate work to reduce the impact of a few very large partitions.

20. When would you still tune partitions manually?

Answer: For predictable workloads, badly sized input partitions, downstream file-size requirements, or when runtime behavior needs explicit control.

5. Joins

21. What is `spark.sql.autoBroadcastJoinThreshold`?

Answer: The maximum estimated size of a relation that Spark can automatically consider for a broadcast join.

22. When should you use `broadcast()` in PySpark?

Answer: When one side of a join is small enough to replicate to executors and broadcasting avoids a large shuffle.

23. What is the downside of broadcasting?

Answer: Broadcasting a relation that is too large can consume executor memory and cause failures or GC pressure.

24. How do you detect a broadcast join?

Answer: Use `explain()`/`explain("formatted")` and inspect the physical plan for `BroadcastHashJoin` or related broadcast operators.

25. Why might Spark not broadcast a table even when you expect it to?

Answer: Statistics may be missing/inaccurate, the table may exceed the threshold, a join type may not support that strategy, or an explicit hint/plan rule may affect selection.

6. Memory, Caching & Serialization

26. `cache()` vs `persist()` in PySpark?

Answer: `cache()` uses the default storage level; `persist()` lets you choose a storage level.

27. When should you cache a `DataFrame`?

Answer: When the same expensive lineage is reused multiple times and the recomputation cost is greater than the memory/storage cost.

28. Why can caching make a job slower?

Answer: It consumes resources, can trigger eviction/recomputation, and adds serialization/storage overhead.

29. What is Kryo serialization?

Answer: A faster and more compact serializer than Java serialization for many JVM-side Spark workloads; PySpark users should understand it mainly when interacting with JVM Spark configuration.

7. PySpark-Specific

30. Why can PySpark be slower than native Spark SQL/`DataFrame` expressions?

Answer: Python/JVM serialization and Python execution overhead can matter. Prefer built-in Spark SQL functions over Python UDFs when possible.

31. Why are built-in functions preferred over Python UDFs?

Answer: Built-in expressions can be optimized by Spark's SQL engine and avoid much of the Python execution/serialization overhead.

32. When is `pandas` UDF useful?

Answer: For vectorized Python processing where built-in functions cannot express the logic and the data can be processed efficiently in batches.

33. What configuration is important for Python workers?

Answer: PySpark has Python/worker-related settings, and executor memory overhead is important because Python worker memory is outside JVM heap.

8. Structured Streaming

34. Which Spark streaming API should you focus on for modern PySpark interviews?

Answer: Structured Streaming. The older DStream-based Spark Streaming API is legacy; Apache recommends Structured Streaming for new streaming applications.

35. What is checkpointLocation?

Answer: A durable location used to store streaming query progress/state information needed for recovery.

36. What is trigger in Structured Streaming?

Answer: It controls when Spark processes available/new data, such as processing-time or available-now styles.

37. Why can stateful streaming require substantial memory?

Answer: Operations such as aggregations, deduplication, joins, and arbitrary stateful processing may retain state across micro-batches.

9. Scenario-Based Questions

38. A 500 GB DataFrame has a groupBy that is very slow. What do you check?

Answer: Check input partitioning, shuffle size, skew, number of shuffle partitions, AQE, executor resources, spill, and whether filters/projections can be pushed earlier.

39. A join has one key with millions of records and most keys are small. What is the issue?

Answer: Data skew. A single shuffle partition can become much larger than others and create a straggler task.

40. The Spark UI shows many tiny tasks. What could be wrong?

Answer: Over-partitioned input, excessive shuffle partitions, small files, or repeated repartitioning.

41. Executors are OOM during a broadcast join. What do you investigate?

Answer: Broadcast relation size, threshold/hints, executor heap, memory overhead, concurrent tasks, and whether broadcasting is actually appropriate.

42. The driver is OOM while executors are mostly idle. What is a likely cause?

Answer: Driver-side collection or materialization such as collect(), toPandas(), large query plans, or excessive driver metadata.

High-Priority Configurations to Memorize

spark.executor.memory — Executor JVM heap

spark.executor.memoryOverhead — Non-heap/Python/native overhead

spark.driver.memory — Driver JVM heap

spark.sql.shuffle.partitions — Default SQL shuffle partition count

spark.default.parallelism — Default parallelism for applicable RDD operations

spark.sql.adaptive.enabled — AQE on/off

spark.sql.autoBroadcastJoinThreshold — Automatic broadcast threshold

spark.sql.broadcastTimeout — Broadcast wait timeout

spark.sql.files.maxPartitionBytes — Maximum bytes considered for a file-based input partition

spark.sql.files.openCostInBytes — Estimated file-opening cost used in file partitioning

spark.task.maxFailures — Maximum failures for a task before the stage is aborted

spark.speculation — Speculative execution

spark.dynamicAllocation.enabled — Dynamic executor allocation

spark.dynamicAllocation.minExecutors — Minimum executors

spark.dynamicAllocation.maxExecutors — Maximum executors

spark.dynamicAllocation.initialExecutors — Initial executor count for dynamic allocation

Source basis: Apache Spark 4.0.1 official configuration and PySpark/Spark SQL documentation. Spark 4.0.1 is used consistently for this handbook; defaults can differ by Spark version or cluster manager.